

SKIVE on gpt-oss-20b: GSM8K, full test set, every lever on both metrics

Campaign run on 2026-09-28 on one NVIDIA L40S (48 GB). All 1,319 GSM8K test problems in every configuration; 71 configurations: FullKV three ways; for each of the two SKIVE metrics and each of the budgets 256, 512 and 1024, the plain run plus seven levers; and, to fill in the budget ladder, budgets 384, 640 and 768 with the three levers that matter (prompt protected, plus slow cadence, plus trajectory and redundancy). Every generated answer is in [skive/results/gsm8k/grid/gsm8k_answers.csv.gz](#) (one row per problem per configuration: question, reference, gold, answer, correct or not, generated tokens; 93,649 rows).

Setup

item	value
model / engine	openai/gpt-oss-20b on vLLM 0.23.0 with SKIVE; TRITON_ATTN attention backend, Marlin MXFP4 experts; max_model_len 32768, gpu_memory_utilization 0.90, prefix caching off, tensor parallel 1
generation	greedy (temperature 0), reasoning_effort=medium, up to 2,048 output tokens; the answer is the last number in the harmony final channel, compared numerically with the reference
prompts	the GSM8K question plus "Please reason step by step and put your final answer after '####.'"; about 100 to 200 tokens
traces	about 450 generated tokens on average under FullKV, so a request's cache is 550 to 650 tokens: a 1024 budget evicts on 8 to 10 percent of requests, 768 on 17, 640 on 26, 512 on about 42, 384 on 71, 256 on all of them
statistics	one problem is 0.08 points; FullKV eager vs graphs differ by 0.2 from run-to-run noise; treat differences under about 0.7 as noise
metrics	vk = vk_ratio (static, cached per block), va = value_attention (query-dependent)
levers (suffixes)	plain : fixed 2-block sink, nothing else; pp : prompt protected (sink = prompt blocks + 1); traj : 16-query trajectory window; red : redundancy penalty 0.3; fp8 : FP8 KV cache; e64 : evict every 64 steps (margin 16) instead of every 16; blk64 : 64-token blocks. All rows use piecewise CUDA graphs and a 128-token local window (8 blocks at size 16; 4 blocks at the 256 budget so the window fits inside the budget; 2 blocks at size 64)
columns	dAcc vs FullKV with graphs; cross% = requests that evicted at least one block; evict rate = cache tokens evicted per generated token; KV saved = evicted tokens over all tokens; rep4g = repeated 4-gram fraction; ttft, tpot, e2e = vLLM per-request stats; wall = batch wall time for all 1,319 problems; gen = mean generated tokens

Verdict

- **At a 512-token budget SKIVE is lossless on GSM8K with value_attention and prompt protection:** 93.03 vs 93.10, with 42 percent of requests evicting and 26 percent of KV reclaimed. Adding the redundancy penalty and the trajectory window gives the best number of the whole grid, 93.63 (+0.5, at the edge of noise), at a decode-speed cost for trajectory (tpot 33 to 39 ms).
- **The budget ladder (256 to 1024, both metrics, table below):** value_attention with prompt protection is within noise of FullKV from 640 tokens up (93.3 at 640 with 17 percent of KV reclaimed, 93.0 at 512 with 26 percent) and loses 1.7 at 384 (91.4, 39 percent reclaimed; 91.9 with slow cadence); vk_ratio is within noise only from 768 up and loses 5.3 at 384 (87.8). The gap between the two metrics opens as the budget drops: +0.4 at 768, +1.0 at 640 and 512, +3.6 at 384, +2.4 at 256.
- **Prompt protection is again the single most important change:** +3.6 points at 512 (89.4 to 93.0) and +8 at 256 (69.3 to 77.2) for value_attention; +6.6 at 256 for vk_ratio. Without it the model loses the question and re-derives (rep4g doubles).
- **At a 256-token budget every request evicts and the budget is below the trace length,** so accuracy falls to 77 to 78 for most value_attention variants and traces grow from 453 to about 790 tokens. The lever that matters there is **slow eviction cadence** (`e64`): 83.4, plus 6 over the base row, with the same 67 to 73 percent of KV reclaimed and the shortest traces of the block. Evicting every 64 steps lets a request run up to 16 blocks over budget between rounds, so the working context of the derivation stays longer and the evictions happen in larger, better-informed batches.
- **value_attention beats vk_ratio wherever eviction is active:** at 512, 93.0 vs 92.0 (base rows) and 93.6 vs 92.3 (best rows); at 256, 77.2 vs 74.8 (base) and 83.4 vs 79.8 (best). At 1024 the two are indistinguishable because almost nothing is evicted.
- **Levers that cost accuracy on this task:** FP8 cache (about -0.8 for FullKV and -0.5 to -2.3 with eviction), 64-token blocks with vk_ratio at 512 (-2.6), and the trajectory window for vk_ratio (no gain, unlike for value_attention). Eviction cadence and blocks are within noise at 512 and 1024 for value_attention.
- **Latency:** with graphs on, SKIVE's per-token decode matches FullKV (tpot 31 to 33 ms vs 31) at 512 and 1024, and wall time is within 4 to 8 percent. At 256 wall time rises with the longer traces, not with the eviction itself. GSM8K never fills the cache at this concurrency, so there is no throughput gain to show here; the value is the memory reclaimed at zero accuracy cost at 512.
- **Recommendation for short-reasoning workloads like GSM8K:** value_attention, prompt protected, redundancy penalty on, budget 512 tokens (lossless, 26 percent reclaimed); if the budget must go below the trace length, add slow cadence (`SKIVE_EVICT_EVERY=64`). Compared with AIME (7,000-token traces, where 4096 was the lossless budget), the lossless budget scales with the trace length, roughly one budget of the mean trace.

Evaluation matrix: budget x {vk_ratio, value_attention}

One line per budget, both metrics side by side, the layout used for the other SKIVE model reports. cross% = share of the 1,319 requests that evicted at least one block (the same for both metrics to within a point; the value_attention figure is shown). acc = exact-match accuracy with the delta vs FullKV with graphs in parentheses. ttft, e2e = mean per-request time to first token and end-to-end time in seconds as reported by vLLM (with 1,319 requests queued at once, ttft is dominated by queueing, so it moves with the batch's total work); wall = wall time for the whole batch; KV saved = evicted tokens over all cache tokens (value_attention row). Percent deltas are vs FullKV with graphs.

prompt protected (pp)

The base SKIVE row: sink = prompt blocks + 1, evict every 16 steps, 128-token local window.

Budget	cross%	vk acc (d)	va acc (d)	vk ttft (s)	va ttft (s)	vk e2e (s)	va e2e (s)	vk wall (s)	va wall (s)	KV saved
FullKV cg	0%	93.10	93.10	28	28	41	41	104	104	-
FullKV cg_fp8	0%	92.27	92.27	27	27	41	41	104	104	-
FullKV eager	0%	92.87	92.87	35	35	53	53	153	153	-
256	99%	74.8 (-18.3)	77.2 (-15.9)	51 (+86%)	55 (+102%)	82 (+97%)	88 (+113%)	188 (+81%)	206 (+98%)	73%
384	71%	87.8 (-5.3)	91.4 (-1.7)	32 (+16%)	30 (+10%)	50 (+20%)	46 (+10%)	123 (+18%)	114 (+10%)	39%
512	40%	92.0 (-1.1)	93.0 (-0.1)	28 (+2%)	28 (+1%)	43 (+4%)	42 (+2%)	110 (+6%)	108 (+4%)	26%
640	26%	92.3 (-0.8)	93.3 (+0.2)	27 (-0%)	29 (+5%)	42 (+2%)	43 (+5%)	108 (+4%)	105 (+1%)	17%
768	17%	92.9 (-0.2)	93.1 (+0.0)	27 (-0%)	29 (+6%)	42 (+1%)	44 (+6%)	106 (+2%)	110 (+5%)	13%
1,024	8%	92.6 (-0.5)	93.6 (+0.5)	27 (-2%)	28 (+1%)	41 (-0%)	42 (+2%)	106 (+2%)	108 (+4%)	8%

prompt protected, evict every 64 steps (pp_e64)

Same, with `SKIVE_EVICT EVERY=64 SKIVE_EVICT_MARGIN=16`: the best lever when the budget is below the trace length.

Budget	cross%	vk acc (d)	va acc (d)	vk ttft (s)	va ttft (s)	vk e2e (s)	va e2e (s)	vk wall (s)	va wall (s)	KV saved
FullKV cg	0%	93.10	93.10	28	28	41	41	104	104	-
FullKV cg_fp8	0%	92.27	92.27	27	27	41	41	104	104	-
FullKV eager	0%	92.87	92.87	35	35	53	53	153	153	-
256	96%	79.8 (-13.3)	83.4 (-9.7)	42 (+54%)	42 (+52%)	66 (+59%)	65 (+56%)	149 (+43%)	148 (+42%)	67%
384	64%	88.1 (-5.0)	91.9 (-1.2)	32 (+18%)	27 (-0%)	50 (+21%)	42 (+1%)	122 (+18%)	107 (+3%)	37%
512	38%	91.3 (-1.8)	92.5 (-0.6)	28 (+2%)	27 (-0%)	43 (+4%)	42 (+0%)	108 (+4%)	105 (+1%)	24%
640	24%	91.7 (-1.4)	93.3 (+0.2)	28 (+2%)	27 (-1%)	43 (+4%)	42 (+0%)	109 (+5%)	104 (+0%)	17%
768	16%	92.9 (-0.2)	93.3 (+0.2)	27 (-1%)	27 (-1%)	42 (+1%)	41 (-0%)	107 (+2%)	104 (+0%)	12%
1,024	8%	92.7 (-0.4)	92.9 (-0.2)	27 (-2%)	27 (-1%)	41 (-0%)	42 (+0%)	106 (+2%)	105 (+1%)	7%

prompt protected, trajectory window, redundancy penalty (pp_traj_red)

Same as `pp` with `SKIVE_QHIST=16 SKIVE_REDUNDANCY=0.3`: the best value_attention numbers at 512 and 640, at a 17 to 24 percent decode-time cost for the 16-query rescoring.

Budget	cross%	vk acc (d)	va acc (d)	vk ttft (s)	va ttft (s)	vk e2e (s)	va e2e (s)	vk wall (s)	va wall (s)	KV saved
FullKV cg	0%	93.10	93.10	28	28	41	41	104	104	-
FullKV cg_fp8	0%	92.27	92.27	27	27	41	41	104	104	-
FullKV eager	0%	92.87	92.87	35	35	53	53	153	153	-
256	99%	73.1 (-20.0)	77.4 (-15.7)	51 (+84%)	54 (+97%)	81 (+96%)	89 (+115%)	189 (+81%)	226 (+117%)	72%
384	72%	87.2 (-5.9)	91.3 (-1.8)	32 (+16%)	34 (+22%)	50 (+20%)	52 (+24%)	123 (+18%)	128 (+23%)	39%
512	42%	91.7 (-1.4)	93.6 (+0.5)	28 (+4%)	33 (+19%)	44 (+6%)	50 (+20%)	111 (+7%)	123 (+18%)	25%
640	25%	92.8 (-0.3)	93.6 (+0.5)	28 (+1%)	33 (+19%)	43 (+3%)	50 (+21%)	108 (+4%)	125 (+20%)	18%
768	17%	92.5 (-0.6)	93.2 (+0.1)	28 (+0%)	32 (+17%)	42 (+2%)	49 (+19%)	107 (+3%)	122 (+17%)	13%
1,024	8%	92.8 (-0.3)	93.2 (+0.1)	27 (-0%)	32 (+16%)	42 (+1%)	49 (+18%)	106 (+2%)	123 (+18%)	8%

no prompt protection (**plain**)

Fixed 2-block sink, for reference: what every SKIVE row looked like before prompt protection. Only 256, 512 and 1024 were run this way.

Budget	cross%	vk acc (d)	va acc (d)	vk ttft (s)	va ttft (s)	vk e2e (s)	va e2e (s)	vk wall (s)	va wall (s)	KV saved
FullKV cg	0%	93.10	93.10	28	28	41	41	104	104	-
FullKV cg_fp8	0%	92.27	92.27	27	27	41	41	104	104	-
FullKV eager	0%	92.87	92.87	35	35	53	53	153	153	-
256	99%	68.2 (-24.9)	69.3 (-23.8)	39 (+43%)	36 (+31%)	61 (+48%)	56 (+36%)	143 (+38%)	137 (+32%)	68%
512	42%	91.7 (-1.4)	89.4 (-3.7)	28 (+2%)	29 (+7%)	43 (+4%)	46 (+10%)	110 (+6%)	116 (+12%)	31%
1,024	9%	93.2 (+0.1)	92.6 (-0.5)	27 (-2%)	28 (+0%)	41 (-0%)	43 (+3%)	106 (+2%)	111 (+6%)	9%

Reading across the four tables: at every budget below 1024, value_attention is the better metric, and the lever that helps depends on where the budget sits relative to the trace length (about 450 generated tokens plus a 100 to 200-token prompt). Above the trace length (640 and up) the plain prompt-protected row is already lossless and nothing else is needed. Around the trace length (512) the trajectory window and redundancy penalty add half a point. Below it (384 and 256) only slow cadence helps, and it also removes the latency overhead (at 384, e2e 42 s vs 41 s for FullKV, against 46 s for the base row).

value_attention: accuracy and latency (all budgets, all levers)

FullKV acc 93.10 (graphs), 92.87 (eager), 92.27 (FP8). Deltas vs FullKV with graphs.

Config	cross%	evict rate	acc (dAcc)	rep4g	tfft (s)	tpot (ms)	e2e (s)	wall (s)	gen tok	batch tok/s	KV saved
FullKV cg	-	-	93.10	0.023	27.5	31	41.4	104	453	5734	-
FullKV eager	-	-	92.87 (-0.23)	0.020	35.1	42	53.5	153	443	3820	-
FullKV cg fp8	-	-	92.27 (-0.83)	0.023	27.0	29	40.7	104	475	6050	-
va @1024 plain	9%	0.12	92.57 (-0.53)	0.030	27.6	33	42.5	111	462	5503	9%
va @1024 pp	8%	0.10	93.56 (+0.45)	0.022	27.7	33	42.4	108	454	5527	8%
va @1024 pp traj	8%	0.11	93.03 (-0.08)	0.023	32.3	38	49.3	123	455	4866	8%
va @1024 pp red	9%	0.11	92.80 (-0.30)	0.020	28.0	33	42.9	109	462	5562	9%
va @1024 pp traj red	8%	0.11	93.18 (+0.08)	0.021	31.9	38	48.9	123	456	4901	8%
va @1024 pp fp8	9%	0.12	92.04 (-1.06)	0.022	28.1	31	42.4	110	471	5647	9%
va @1024 pp e64	8%	0.10	92.87 (-0.23)	0.021	27.3	32	41.6	105	452	5671	7%
va @1024 pp blk64	8%	0.11	92.19 (-0.91)	0.022	28.8	33	43.3	110	453	5429	8%
va @768 pp	17%	0.17	93.10 (+0.00)	0.019	29.3	33	43.9	110	450	5413	13%
va @768 pp e64	16%	0.16	93.33 (+0.23)	0.022	27.2	32	41.4	104	451	5692	12%
va @768 pp traj red	17%	0.17	93.18 (+0.08)	0.022	32.3	38	49.3	122	454	4896	13%
va @640 pp	26%	0.22	93.33 (+0.23)	0.019	28.9	33	43.3	105	444	5564	17%
va @640 pp e64	24%	0.22	93.25 (+0.15)	0.019	27.3	32	41.6	104	455	5754	17%
va @640 pp traj red	25%	0.24	93.63 (+0.53)	0.020	32.8	39	50.0	125	457	4831	18%
va @512 plain	42%	0.39	89.39 (-3.71)	0.051	29.5	33	45.6	116	499	5649	31%
va @512 pp	40%	0.33	93.03 (-0.08)	0.020	27.7	33	42.4	108	455	5541	26%
va @512 pp traj	42%	0.33	93.18 (+0.08)	0.022	32.5	39	49.8	125	457	4806	26%
va @512 pp red	42%	0.33	93.25 (+0.15)	0.019	28.2	33	42.9	108	455	5547	25%
va @512 pp traj red	42%	0.33	93.63 (+0.53)	0.020	32.8	39	49.9	123	452	4865	25%
va @512 pp fp8	44%	0.34	92.57 (-0.53)	0.020	28.5	31	42.6	108	463	5656	26%

value_attention, 384 and 256-token budgets (below the trace length)

Config	cross%	evict rate	acc (dAcc)	rep4g	ttft (s)	tpot (ms)	e2e (s)	wall (s)	gen tok	batch tok/s	KV saved
va @384 pp	71%	0.51	91.36 (-1.74)	0.023	30.3	33	45.6	114	472	5446	39%
va @384 pp e64	64%	0.47	91.89 (-1.21)	0.022	27.4	32	41.9	107	470	5805	37%
va @384 pp traj red	72%	0.51	91.28 (-1.82)	0.025	33.7	39	51.6	128	472	4864	39%
va @256 plain	99%	0.82	69.29 (-23.81)	0.115	35.9	32	56.4	137	657	6325	68%
va @256 pp	99%	0.86	77.18 (-15.92)	0.049	55.5	41	88.5	206	801	5130	73%
va @256 pp traj	99%	0.86	77.26 (-15.85)	0.054	56.1	45	92.1	230	786	4498	73%
va @256 pp red	99%	0.85	78.32 (-14.78)	0.050	44.6	37	72.7	182	757	5484	72%
va @256 pp traj red	99%	0.85	77.41 (-15.69)	0.054	54.1	45	88.9	226	759	4420	72%
va @256 pp fp8	100%	0.86	77.86 (-15.24)	0.054	46.8	35	74.7	184	787	5647	73%
va @256 pp e64	96%	0.80	83.40 (-9.70)	0.034	41.7	33	64.6	148	701	6265	67%

vk_ratio: accuracy and latency (all budgets, all levers)

Deltas vs FullKV with graphs (93.10).

Config	cross%	evict rate	acc (dAcc)	rep4g	tfft (s)	tpot (ms)	e2e (s)	wall (s)	gen tok	batch tok/s	KV saved
vk @1024 plain	8%	0.11	93.18 (+0.08)	0.020	27.0	32	41.4	106	457	5691	8%
vk @1024 pp	8%	0.11	92.57 (-0.53)	0.022	27.0	32	41.4	106	459	5680	9%
vk @1024 pp traj	8%	0.11	92.87 (-0.23)	0.019	27.4	32	41.8	106	456	5674	8%
vk @1024 pp red	10%	0.13	92.72 (-0.38)	0.024	27.6	32	42.5	109	472	5700	10%
vk @1024 pp traj red	9%	0.12	92.80 (-0.30)	0.022	27.4	32	42.0	106	461	5716	9%
vk @1024 pp fp8	9%	0.11	92.12 (-0.99)	0.019	26.3	30	40.2	103	473	6055	9%
vk @1024 pp e64	8%	0.11	92.72 (-0.38)	0.023	26.9	32	41.3	106	459	5728	8%
vk @1024 pp blk64	9%	0.12	92.95 (-0.15)	0.019	27.1	32	41.6	106	460	5734	9%
vk @768 pp	16%	0.18	92.95 (-0.15)	0.020	27.4	32	41.7	106	455	5665	13%
vk @768 pp e64	16%	0.19	92.87 (-0.23)	0.023	27.1	32	41.7	107	468	5790	15%
vk @768 pp traj red	17%	0.19	92.49 (-0.61)	0.021	27.6	32	42.2	107	461	5692	14%
vk @640 pp	26%	0.25	92.27 (-0.83)	0.020	27.5	32	42.1	108	465	5677	19%
vk @640 pp e64	23%	0.28	91.66 (-1.44)	0.024	28.0	32	43.1	109	489	5903	21%
vk @640 pp traj red	26%	0.26	92.80 (-0.30)	0.022	27.8	32	42.5	108	470	5748	20%
vk @512 plain	41%	0.37	91.66 (-1.44)	0.028	28.1	32	43.1	110	480	5726	28%
vk @512 pp	42%	0.37	91.96 (-1.14)	0.019	28.1	32	43.3	110	485	5797	29%
vk @512 pp traj	41%	0.38	91.96 (-1.14)	0.024	28.3	32	43.6	111	493	5851	30%
vk @512 pp red	43%	0.36	92.34 (-0.76)	0.021	28.0	32	43.0	108	479	5839	28%
vk @512 pp traj red	42%	0.38	91.66 (-1.44)	0.022	28.5	32	43.8	111	490	5813	29%
vk @512 pp fp8	44%	0.39	90.83 (-2.27)	0.023	27.3	30	42.0	107	505	6236	31%
vk @512 pp e64	37%	0.37	91.28 (-1.82)	0.021	28.1	31	43.1	108	493	6017	28%
vk @512 pp blk64	42%	0.44	90.52 (-2.58)	0.024	29.3	31	45.5	115	528	6068	35%

vk_ratio, 384 and 256-token budgets (below the trace length)

Config	cross%	evict rate	acc (dAcc)	rep4g	ttft (s)	tpot (ms)	e2e (s)	wall (s)	gen tok	batch tok/s	KV saved
vk @384 pp	71%	0.60	87.79 (-5.31)	0.031	32.0	31	49.8	123	581	6238	49%
vk @384 pp e64	64%	0.58	88.10 (-5.00)	0.026	32.4	31	50.2	122	593	6388	47%
vk @384 pp traj red	72%	0.60	87.19 (-5.91)	0.032	31.9	31	49.8	123	583	6242	49%
vk @256 plain	99%	0.84	68.16 (-24.94)	0.095	39.5	31	61.3	143	728	6698	70%
vk @256 pp	99%	0.87	74.75 (-18.35)	0.058	51.3	33	81.5	188	904	6336	76%
vk @256 pp traj	99%	0.88	72.63 (-20.47)	0.058	52.4	33	83.6	194	930	6305	76%
vk @256 pp red	99%	0.88	73.54 (-19.56)	0.058	52.5	33	83.6	193	927	6334	76%
vk @256 pp traj red	99%	0.87	73.09 (-20.02)	0.059	50.7	33	81.0	189	910	6367	76%
vk @256 pp fp8	99%	0.88	73.09 (-20.02)	0.059	50.4	32	80.1	186	921	6518	76%
vk @256 pp e64	96%	0.82	79.76 (-13.34)	0.035	42.4	30	65.8	149	797	7073	70%

vk_ratio vs value_attention, best row per budget

At 384, 640 and 768 only `pp`, `pp_e64` and `pp_traj_red` were run, so "best" is over those three there and over all eight levers elsewhere.

budget	best vk (config)	best va (config)	base vk (pp)	base va (pp)	KV saved (pp)
1024	93.18 (plain)	93.56 (pp)	92.57	93.56	8 to 9%
768	92.95 (pp)	93.33 (pp e64)	92.95	93.10	13%
640	92.80 (pp traj red)	93.63 (pp traj red)	92.27	93.33	17 to 19%
512	92.34 (pp red)	93.63 (pp traj red)	91.96	93.03	26 to 29%
384	88.10 (pp e64)	91.89 (pp e64)	87.79	91.36	39 to 49%
256	79.76 (pp e64)	83.40 (pp e64)	74.75	77.18	73 to 76%

How this compares with AIME 2024

Same model, same levers, different trace length. On AIME (7,000-token traces) the lossless budget was 4096 tokens and prompt protection recovered

16 points at 2048; on GSM8K (450-token traces) the lossless budget is 512 and prompt protection recovers 4 to 8 points. In both, value_attention beats vk_ratio wherever eviction is active, FP8 with eviction costs a little accuracy, and 64-token blocks trade accuracy for eviction count. The new observation here is the cadence lever: when the budget is below the trace length, evicting every 64 steps instead of 16 is worth 5 to 6 points on both metrics, which did not show on AIME because there the budget was above most of the trace.

